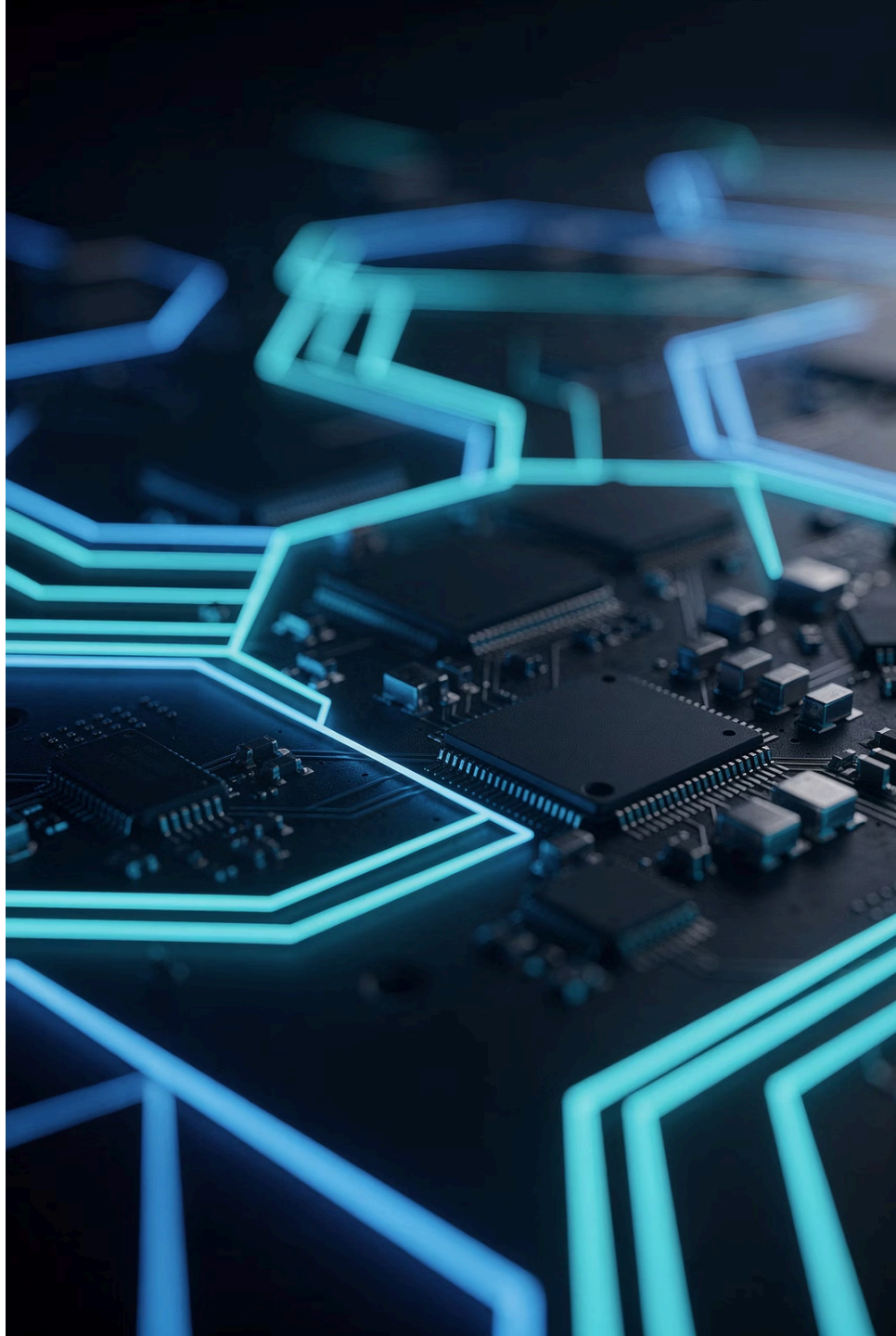
Module 5: Signals & Angular Reactivity

Modern State Management in Angular 21

ANGULAR 21 MODULE 5



Module Overview

By the end of this module, you will be able to:

01

Reactivity Model

Understand Angular's core reactivity model and how it drives UI updates

02

Signals

Explain Signals, their benefits, and differentiate Writable vs. Computed types

03

Effects & Inputs

Use Effects for side effects and Signal Inputs & Outputs for component communication

04

Linked Signals

Manage dependent state with linkedSignal() and apply development best practices

Why Angular Reactivity?

Modern applications constantly change data based on user interactions, API responses, and business events. Angular's reactive model keeps the UI automatically synchronized with state — no manual DOM updates required.

Dashboard Updates

Shopping Cart

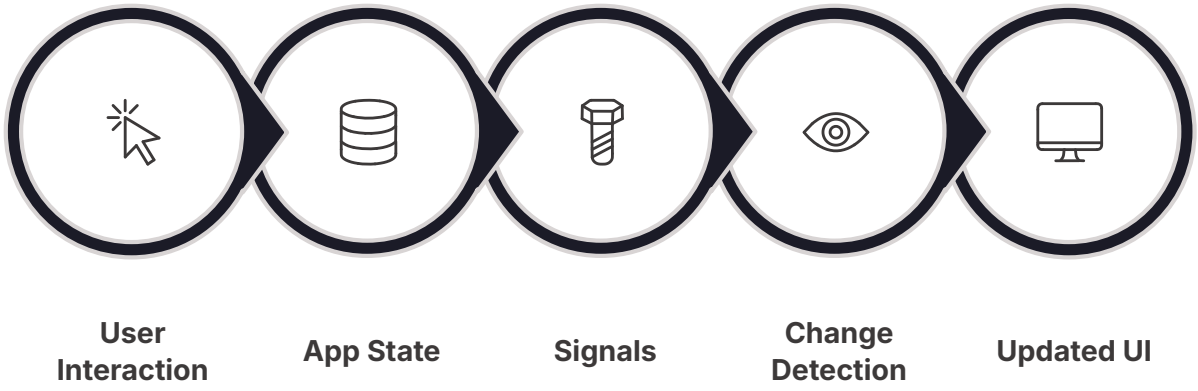
Search Filters

Form Validation

Notifications

Key Point: Reactivity keeps the UI synchronized with data without manually updating the DOM.

Angular Reactivity Model



Angular's pipeline ensures every state change flows predictably from user action to rendered output.

Why This Model Wins

Automatic UI Updates

No imperative DOM calls needed

Predictable State

One-way data flow keeps bugs isolated

Better Performance

Only affected nodes re-render

Simplified Logic

Less glue code in your components

What Are Signals?

A **Signal** is a reactive container that stores a value and automatically notifies Angular when that value changes — enabling fine-grained, efficient UI updates.



Holds State

Wraps any application value — primitives, objects, arrays



Tracks Changes

Automatically notifies consumers when the value updates



Triggers Re-render

Angular re-renders only the affected parts of the UI



Easy to Maintain

Replaces manual change detection with readable, declarative code

Common Use Cases

- Counters & toggles
- User profile data
- Theme & language selection
- Shopping cart contents
- Real-time dashboard metrics



Writable Signals

Writable Signals are the foundation of mutable application state. They store a value that can be updated at any time, automatically notifying all dependent components.

Mutable Values

Store and update any piece of state dynamically

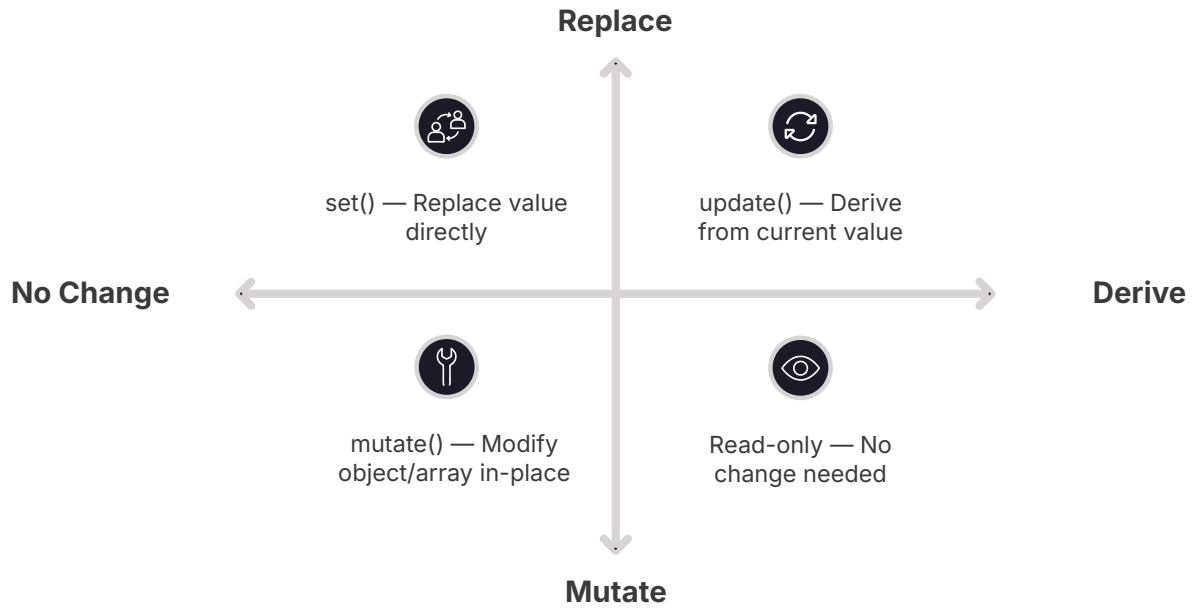
Instant Notification

Dependent components re-render automatically on change

Typical Examples

Counter, logged-in user, language, cart quantity, form state

Updating Signals



Choose the right method based on whether you are replacing, deriving, or mutating the current value.

Common Triggers

Button Click

Increment a counter or toggle a flag

API Response

Populate a user profile or refresh a list

Form Input

Reflect live text as the user types

Theme / Login Change

Switch global UI state in one signal update

Computed Signals

Computed Signals derive new values from one or more existing Signals. They are **read-only**, automatically recalculated whenever their dependencies change, and eliminate duplicate state.



Angular recalculates the computed value only when a dependency signal changes — never unnecessarily.

Benefits & Use Cases

Total Price

Sum items + tax + shipping

Full Name

Combine first + last name signals

Discount

Derive savings from cart total

Dashboard Stats

Aggregate metrics from raw data signals

Effects

Effects execute side-effect logic automatically whenever a Signal they depend on changes. They bridge reactive state and the outside world.

API Calls

Fetch fresh data when a filter signal changes

Local Storage

Persist user preferences on every state change

Analytics

Fire tracking events reactively without manual wiring

External Sync

Keep third-party services in sync with app state

Remember: Effects are for side effects only — never use them to calculate or derive state.



Signal Inputs & Outputs

Angular 21 replaces the traditional `@Input()` and `@Output()` decorators with reactive Signal-based equivalents, producing cleaner APIs and eliminating boilerplate.

Signal Inputs

- Receive reactive data from parent components
- Automatically update when parent state changes
- Fully type-safe and composable

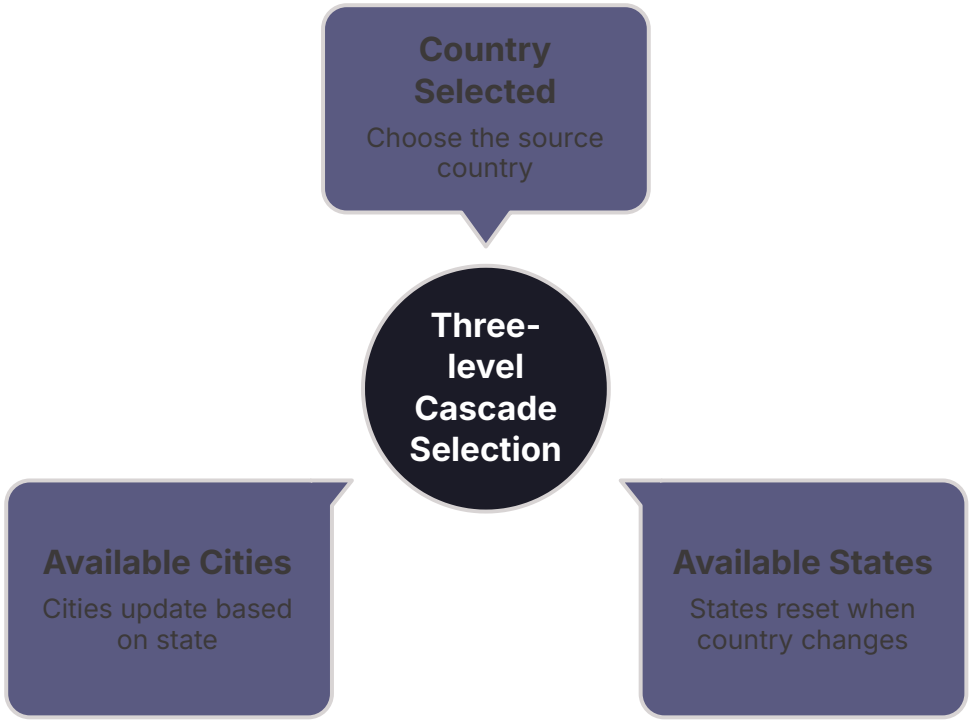
Signal Outputs

- Notify parent components about child events
- Simplify event communication patterns
- Drastically reduce boilerplate code



Linked Signals

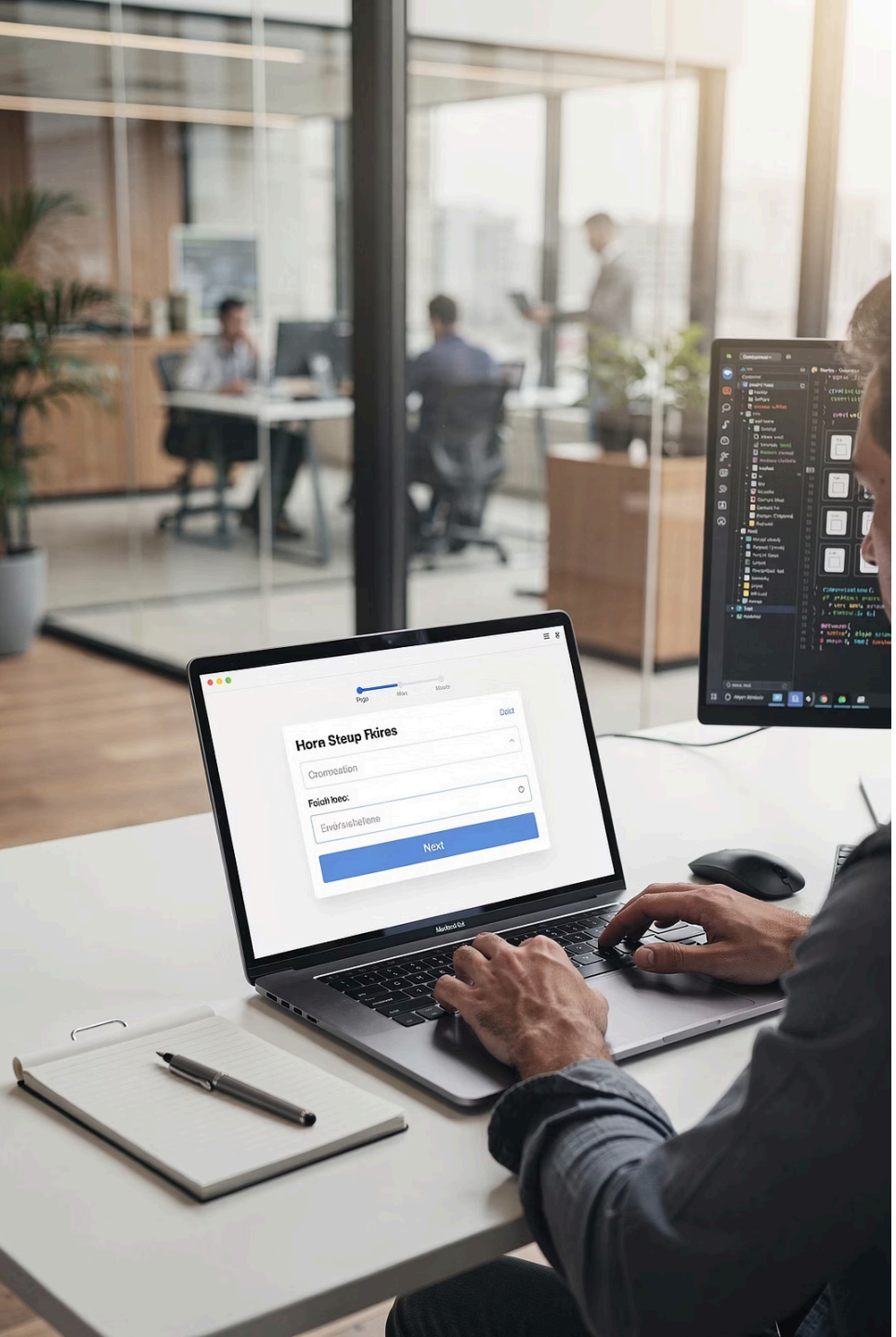
Linked Signals manage state that depends on another Signal. When the source changes, the linked Signal resets automatically — keeping related state consistent without manual coordination.



Real-World Cascades

- → **Country → State → City**
- → **Category → Product**
- → **Department → Employees**
- → **Brand → Models**

Selecting a new country automatically resets the states and cities — zero manual cleanup required.



Managing Dependent State with `linkedSignal()`

`linkedSignal()` creates a writable signal whose value resets whenever its source signal changes — perfect for cascading selections and multi-step workflows.

Cascading Dropdowns

City resets when country changes

Product Config

Options update based on selected variant

Multi-Step Forms

Later steps reset when earlier answers change

Dynamic Filters

Sub-filters clear when parent filter is updated

Signals vs. RxJS

Signals and RxJS are **complementary**, not competing. Understanding when to use each is essential for well-architected Angular applications.



Practical Recommendation

Use Signals for

Application state, UI state, component data, and derived values

Use RxJS for

HTTP requests, WebSockets, complex event streams, and async pipelines

Best Practices

1

Keep Signals Focused

One signal per logical piece of state — avoid overloading a single signal with mixed concerns

2

Computed for Derivations

Never duplicate state — use `computed()` to derive values from existing signals

3

Effects for Side Effects Only

Do not calculate state inside effects — use them for logging, storage, and external sync

4

Combine Signals + RxJS

Use `toSignal()` and `toObservable()` to bridge both paradigms seamlessly

Keep state simple and predictable — the fewer moving parts, the easier debugging becomes.



Module Summary

What We Covered

01

Reactivity Model & Signals

Writable, Computed, Effects

02

Signal Inputs & Outputs

Component communication patterns

03

Linked Signals

Dependent state with linkedSignal()

04

Best Practices

Signals + RxJS working together

Up Next: Module 6

Component Communication & Content Projection

- Parent–Child Communication
- Shared State patterns
- Content Projection
- Communication Patterns